DATA 442/642 · Advanced Machine Learning · Week 15

15.1 Introduction to GANs

A generator and a discriminator playing a two-player game.

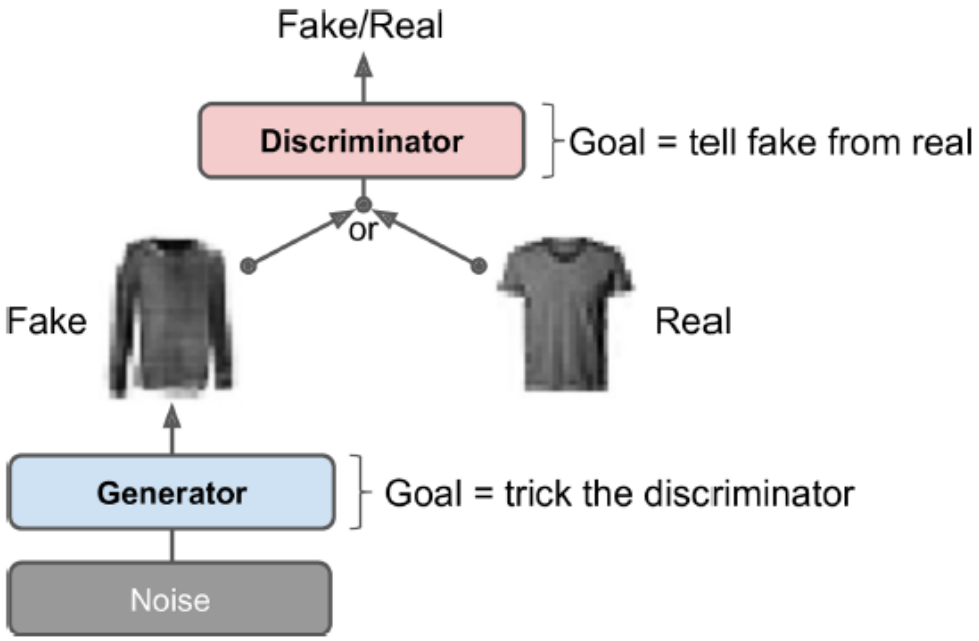
These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

Figures and notes follow Géron, *Hands-On Machine Learning*; Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*; and Theodoridis, *Machine Learning: A Bayesian and Optimization Perspective*.

1. Generative adversarial networks

Generative adversarial networks (GANs) are from Goodfellow et al., 2014. The idea caught on immediately; stable training took years.

A GAN is two neural nets.



2. Applications

Use	Sketch
Image generation and editing	Faces, objects, landscapes; inpainting missing patches.
Data augmentation	Extra synthetic rows when labels are expensive.
Text-to-image	A sentence in, a matching scene out.
Style transfer	Copy the look of one image onto another.
Molecular generation	New drug-like or materials structures.
Anomaly detection	Train on normal data; flag points far from that law.
Domain adaptation	Map a source domain onto a target domain and keep the meaning.

---

3. Generator

The **generator** takes a random draw (usually Gaussian) and emits a sample, often an image.

Treat that draw as a **latent coding**. Functionally this is the decoder of a VAE: Gaussian noise in, a new image out. The training rule is different.

Write z for the noise and

$x = G(z;\theta_g)$

for a fake sample with the same dimension as a real observation x_n, $n = 1, \ldots, N$. θ_g are the generator weights. Training aims to make the fakes statistically indistinguishable from the reals.

---

4. Discriminator

The **discriminator** sees either a fake from G or a real training image, and guesses which. It is a binary classifier. On input x it returns

$y = D(x;\theta_d)$,

the probability that x is real. Then $1 - D(x;\theta_d)$ is the probability it is fake. θ_d are the discriminator weights.

---

5. Training

Opposite goals. The discriminator wants to tell fakes from reals. The generator wants fakes that fool the discriminator. You cannot train this as one ordinary net. Each iteration has two phases.

1. **Train the discriminator.** A batch of reals, plus as many fakes from the current generator. Labels: 0 fake, 1 real. One binary-cross-entropy step. Backprop updates **only** θ_d.

2. **Train the generator.** A fresh batch of fakes. No reals. Every label is 1 (real): you want the discriminator to be *wrong*. Freeze θ_d; backprop updates **only** θ_g.

---

6. The cost function

A two-player min-max game:

$\min_{\theta_g} \max_{\theta_d} J(\theta_g, \theta_d)$,

$J(\theta_g, \theta_d) = \mathbb{E}_{x \sim p_r(x)}\left[\ln D(x;\theta_d)\right] + \mathbb{E}_{z \sim p_z(z)}\left[\ln\left(1 - D(G(z;\theta_g);\theta_d)\right)\right]$.

$p_r(x)$ is the real-data law. With θ_g held fixed, the discriminator is trained so that $D(x;\theta_d)$ is large on reals and $1 - D(G(z;\theta_g);\theta_d)$ is large on fakes.

---

7. Algorithm

Initialize $\theta_d^{(0)}$ and $\theta_g^{(0)}$. Minibatch size K. Let m be the number of discriminator steps per generator step ($m = 1$ in the original paper).

While θ_d and θ_g have not converged:

- For $t = 1, \ldots, m$:
- Sample $z^{(i)}$, $i = 1, \ldots, K$, from $p_z(z)$.
- Sample $x^{(i)}$, $i = 1, \ldots, K$, from $p_r(x)$.
- Ascend on θ_d using

$$\nabla_{\theta_d}\left\{\frac{1}{K}\sum_{i=1}^{K}\left(\ln D(x^{(i)};\theta_d) + \ln\left(1 - D(G(z^{(i)};\theta_g);\theta_d)\right)\right)\right\}.$$

- Then sample a new $z^{(i)}$, $i = 1, \ldots, K$, from $p_z(z)$ and **descend** on θ_g using

$$\nabla_{\theta_g}\left\{\frac{1}{K}\sum_{i=1}^{K}\ln\left(1 - D(G(z^{(i)};\theta_g);\theta_d)\right)\right\}.$$

Practice

1. What two networks are playing the GAN game, and what does each want?
2. In the generator step, why are all labels set to 1 (real), and which weights stay frozen?
3. With θ_g held fixed, what is the discriminator maximizing on real x and on fake $G(z)$?